Projet du jeu *six qui prend*

P. Albers

20 octobre 2026

Le but de ce projet est d'implémenter la version électronique du jeu de plateau créé par Wolfgang Kramer : *le six qui prend* de la société *Gigamic*. L'objectif du projet est uniquement *à but* éducatif, et ne pourra en aucun cas déboucher sur une utilisation commerciale ou non.

1 Environnement de programmation

Le projet se fera sous l'environnement de développement intégré comme *VS Code* ou *eclipse*.

Travail à faire :

1. Lancez votre éditeur de développement.
2. Créez un nouveau projet *sixQuiPrend* (menu *File>New*).
3. Créez un nouveau paquetage *modele* dans ce projet, puis une second paquetage *test*.
4. Téléchargez depuis *Moodle* le fichier *fichiersTestProjetSixQuiPrend* contenant les fichiers tests de l'application, et décompaquetez le dans un répertoire temporaire sur votre machine.
5. Glissez-déposez tous les fichiers de tests dans le paquetage *test*.

2 Le modèle

Le but de cette partie est d'implémenter le diagramme de classe présenté dans la figure 1.

2.1 La classe *Carte*

Travail à faire :

1. Dans le paquetage *modele*, ajoutez la classe *Carte*.
2. Déclarez les deux attributs *valeur* et *nombreTetesDeBoeuf* de la classe, conformément au diagramme de classe (voir figure 1).
3. Ajoutez la méthode privée *calculNbTetes*() qui renvoie le nombre de têtes correspondant au numéro de la carte (voir le description du jeu).

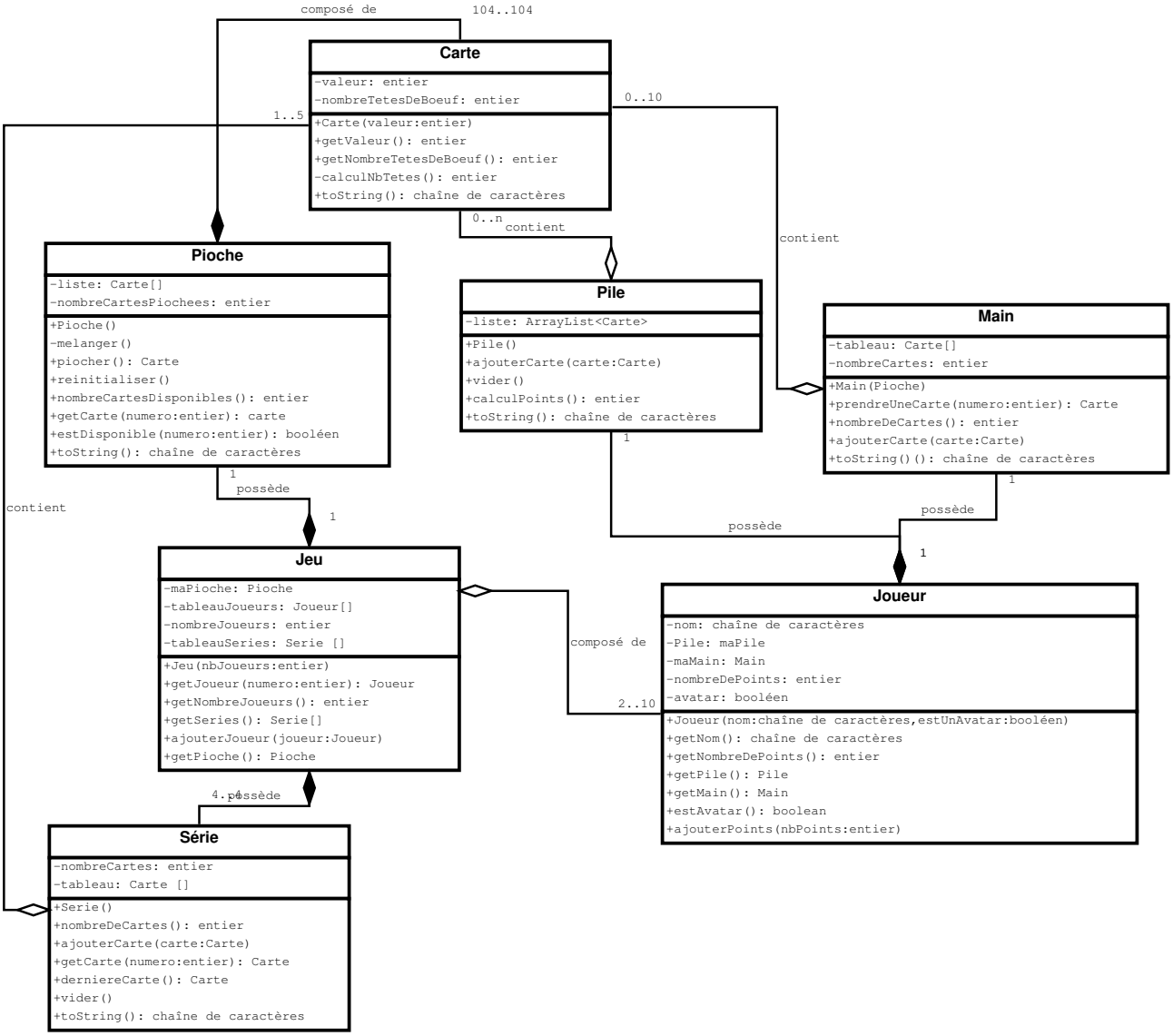


FIGURE 1 – Diagramme de classe du modèle

4. Implémentez le constructeur de la classe qui prendra en paramètre la valeur de la carte. Le constructeur appelera la méthode *calculNbTetes*() pour initialiser l'attribut correspondant.
5. Ajoutez les accesseurs $getValeur()$ et $getNombreTetesDeBoeuf()$.
6. Ajoutez la méthode *toString*() qui renverra une chaîne de caractères au format : $(valeur, nbTetes)$
7. Exécutez la classe $TestCarte$ et décommentez chaque appel aux différents tests (dans la méthode *main*).

   N'oubliez pas de complétez le cahier de tests à chaque test.

2.2 La classe *Main*

La classe *Main* correspond à l'ensemble des cartes contenues dans la main d'un joueur. Le nombre de cartes est compris entre 0 et 10 compris. On utilisera un tableau de *Carte* pour représenter cette main.

Les cartes seront ajoutées une à une dans la main au début de chaque manche. La méthode *prendreUneCarte*(*int i*) retirera la $i^{\text{ème}}$ carte de la main et la renverra. Si l'indice est incorrect, la méthode renverra la valeur *null*.

Travail à faire :

1. Dans le paquetage *modele*, ajoutez la classe *Main*.
2. Ajoutez les attributs *nombreCartes* et *tableau* comme indiqué dans la figure 1.
   L'attribut *tableau* servira à stocker les cartes de la main d'un joueur. Le nombre de cartes sera au maximum de 10 cartes ; la taille du tableau sera donc de 10. L'ordre des cartes n'aura aucune importance.
   L'attribut *nombreCartes* indiquera le nombres de cartes de la main ; au départ, il n'y aura aucune carte dans la main.
3. Implémentez le constructeur *Main*() qui initialisera les deux attributs.
4. Ajoutez l'accesseur en lecture *nombreDeCartes*() qui renverra le nombre de cartes de la main.
5. Implémentez la méthode *ajouterCarte*(*Carte c*) qui ajoutera la carte *c* à la main. Si la main est complète, la méthode n'aura aucun effet.
6. Ajoutez la méthode *prendreUneCarte*(*int i*) qui renverra la $i^{\text{ème}}$ carte de la main. Cette carte sera retirée de la main.
   La méthode devra gérée que la $i^{\text{ème}}$ carte existe bien.
7. Ajoutez enfin la méthode *toString*() qui renverra une chaîne au format suivant : $[(10, 3), (11, 5), (12, 1)]$, si la main est composée des cartes $(10, 3)$, $(11, 5)$ et $(12, 1)$.
8. Exécutez la classe *TestMain* et décommentez chaque appel aux différents tests (dans la méthode *main*).
   N'oubliez pas de complétez le cahier de tests à chaque test.

2.3 La classe *Serie*

La classe *Serie* permettra de stocker les cartes disposées sur la table devant les joueurs. Il y aura 4 séries où les joueurs pourront ajouter leurs cartes.

Comme pour la classe *Main*, la classe *Serie* aura deux attributs ; le nombre de cartes et le tableau contenant les cartes. Il y aura 5 cartes maximum par série.

Travail à faire :

1. Dans le paquetage *modele*, ajoutez la classe *Serie*.
2. Ajoutez les attributs *nombreCartes* et *tableau* comme indiqué dans la figure 1.
   Le nombre de cartes sera au maximum de 5 cartes ; au départ, il n'y aura aucune carte dans la série.
3. Implémentez les méthodes suivantes :
   - le constructeur *Serie*(),
   - l'accesseur *nombreDeCartes*() qui renverra le nombre de cartes de la série,
   - l'accesseur *getCarte*(*int i*) qui renverra la $i^{\text{ème}}$ carte,
   - l'accesseur *derniereCarte*() qui renverra la dernière carte ajoutée,
   - la méthode *ajouterCarte*(*Carte c*) qui ajoute une carte dans l'attribut *tableau*,
     L'ordre d'arrivée sera respectée. La méthode vérifiera que la série n'est pas complète, avant d'ajouter la carte. On pourra également vérifier que la valeur de la carte ajoutée est bien supérieure à celle de la dernière carte de la série.
   - la méthode *vider*() qui remettra le nombre de carte à zéro,
   - la méthode *toString*() qui renverra un String au format : $[(10, 3), (11, 5), (12, 1)]$, si la série est composée des cartes $(10, 3)$, $(11, 5)$ et $(12, 1)$.
4. Exécutez la classe *TestSerie* et décommentez chaque appel aux différents tests (dans la méthode *main*).
   N'oubliez pas de complétez le cahier de tests à chaque test.

3 La classe *Pile*

Cette classe permettra de stocker les cartes qu'un joueur doit récupérer des cartes d'une série. À la fin de la manche, il faudra calculer la somme de toutes les têtes de bœuf des cartes de la pile.

Comme l'on ne connait pas le nombre de cartes que la pile contiendra, nous utiliserons un attribut de type *ArrayList<Carte>* [1].

Travail à faire :

1. Dans le paquetage *modele*, ajoutez la classe *Pile*.
2. Ajoutez l'attribut *liste*, qui sera initialement vide.
3. Implémentez les méthodes suivantes :
   - le constructeur de la classe *Pile*,
   - la méthode *ajouterCarte*(*Carte c*) qui ajoute une carte dans l'attribut *liste*,
   - la méthode *vider*() qui remettra le nombre de supprimer toutes les cartes,
   - la méthode *toString*() qui renverra un String au format : $(10, 3), (11, 5), (12, 1)$, si la série est composée des cartes $(10, 3)$, $(11, 5)$ et $(12, 1)$,
   - la méthode *calculPoints*() qui renverra le nombre de têtes de bœuf des cartes de la pile.
4. Exécutez la classe *TestPile* et décommentez chaque appel aux différents tests (dans la méthode *main*).
   N'oubliez pas de complétez le cahier de tests à chaque test.

1. Voir la description de la classe dans la documentation *Java*.

4 La classe *Pioche*

C'est dans la classe *Pioche* que seront définies toutes les cartes du jeu. Le constructeur de cette classe devra créer toutes les instances des 104 cartes. Les cartes seront créées dans un premier temps dans l'ordre des valeurs des cartes (de 1 à 104) et seront stockées dans un tableau de taille 104. Les cartes seront ensuite mélangées.

On piochera les cartes dans l'ordre en commençant par la première case du tableau. Les cartes resteront dans le tableau, même lorsqu'elle seront piochées. Par conséquent, afin de connaître le nombre de cartes déjà piochées, nous utiliserons l'attribut *nombreCartesPiochees* qui sera initialisé à 0 à chaque début de manche.

Travail à faire :

1. Dans le paquetage *modele*, ajoutez la classe *Pioche*.
2. Ajoutez les attributs *liste* et *nombreCartesPiochees*.
3. Implémentez les méthodes suivantes :
   - le constructeur de la classe *Pioche*, qui créera toutes les cartes du jeu.
   - la méthode privée *melanger*() qui effectuera 1000 échanges dans le tableau ; les indices seront générés aléatoirement [a]. Ajoutez l'appel à cette méthode *melanger*() à la fin du constructeur.
   - l'accesseur *nombreCartesDisponibles*() qui renvoie le nombre de cartes que l'on peut encore piochées,
   - l'accesseur *getCarte*(*int i*) qui renverra la $i^{\text{ème}}$ carte du tableau,
   - l'accesseur *estDisponible*(*int i*) qui renverra si la $i^{\text{ème}}$ carte a déjà été piochée ou non,
   - la méthode *reinitialiser*() qui mélangera de nouveau la pioche et mettra le nombre de cartes piochées à 0. Cette méthode sera appelée à chaque début de manche,
   - et la méthode *piocher*() qui renverra la prochaine carte.
4. Exécutez la classe *TestPioche* et décommentez chaque appel aux différents tests (dans la méthode *main*).

   N'oubliez pas de complétez le cahier de tests à chaque test.

a. Utilisez la classe *Random*.

5 La classe *Joueur*

Un joueur a un nom, est possédera un nombre de points, une main et une pile de cartes. Cette classe servira a représenté l'utilisateur ainsi que des joueurs fictifs, car le but du jeu est de faire jouer un utilisateur contre l'ordinateur. L'application devra donc simuler différents avatars.

Nous ajouterons par conséquent un booléen *avatar* qui indiquera si le joueur est réel ou non.

Travail à faire :

1. Dans le paquetage *modele*, ajoutez la classe *Pioche*.
2. Ajoutez les attributs *nom*, *maPile*, *maMain*, *nombreDePoints* et *avatar*.
3. Implémentez les méthodes suivantes :
   - le constructeur de la classe *Joueur* qui prendra le nom du joueur et la valeur du booléen *avatar* ; il créera une instance de pile et une instance de main, et initialisera le nombre de points à 0,
   - les accesseurs *getNom*(), *getNombreDePoints*(), *getMain*(), *getPile*() et *estAvatar*(),
   - la méthode *ajouterPoints*(*int p*) qui ajoutera *p* au nombre de points.
4. Exécutez la classe *TestJoueur* et décommentez chaque appel aux différents tests (dans la méthode *main*).
   N'oubliez pas de complétez le cahier de tests à chaque test.

6 La classe *Jeu*

La classe *Jeu* comportera la pioche, un tableau de 2 à 10 joueurs maximum, et 4 séries. Pour indiquer le nombre de joueurs, on ajoutera l'attribut *nombreJoueurs*.

Travail à faire :

1. Dans le paquetage *modele*, ajoutez la classe *Jeu*.
2. Ajoutez les attributs *maPioche*, *tableauJoueurs*, *nombreJoueurs* et *tableauSeries*.
3. Implémentez les méthodes suivantes :
   - le constructeur de la classe *Jeu* créera une instance de pioche, un tableau de joueurs de taille 10 et tableau de série de taille 4, ainsi que les instances de 4 qu'il mettra dans le tableau des séries. Le nombre de joueurs sera initialiser au départ à 0,
   - les accesseurs *getJoueur*(*int i*), *getNombreJoueurs*(), *getPioche*(), et *getSerie*(*int i*),
   - la méthode *ajouterJoueur*(*Joueur j*) qui ajoutera le joueur *j* au tableau des joueurs.

7 L'application

La classe *Application* se trouvera dans le paquetage par défaut du projet.

Travail à faire :

1. Téléchargez le fichier *Application.java* est ajoutée le fichier dans le paquetage par défaut.
2. Complétez les méthodes au fur et à mesure.